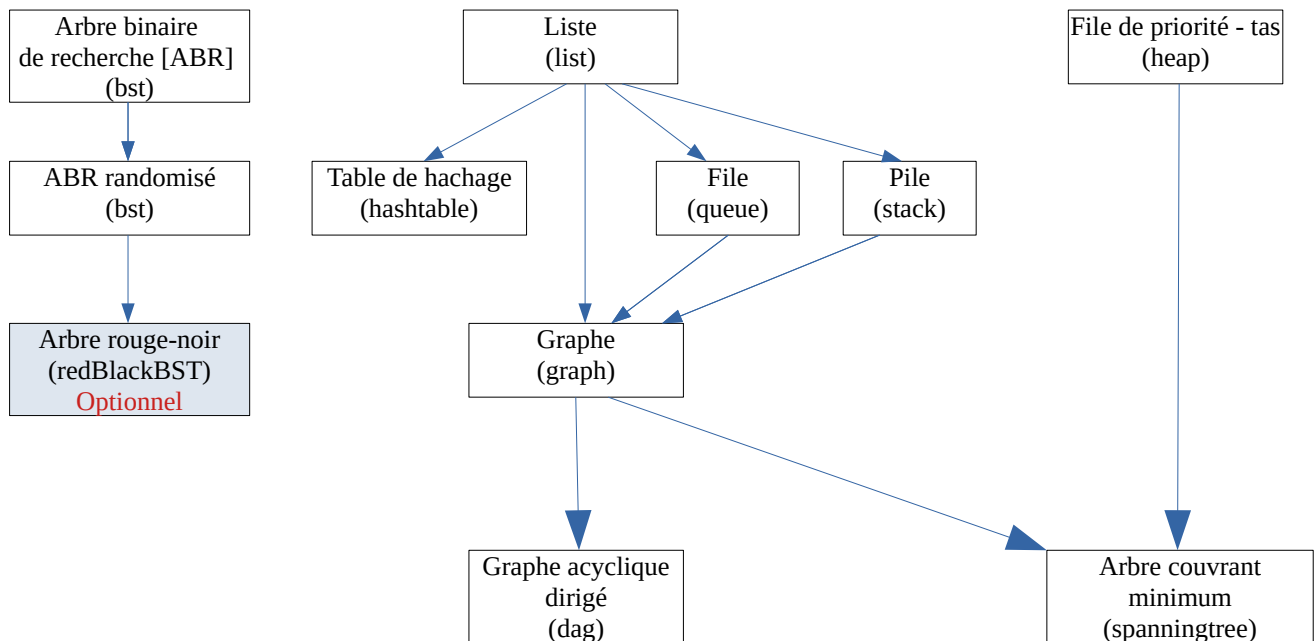


## Cahier de TP Consignes générales

Ce cahier de TP contient tous les sujets de TP liés au cours d'algorithmique avancée. Les Tps sont à réaliser **en monôme**. Chaque monôme est libre d'aller à la vitesse qu'il entend.

Les Tps ont été pensés pour être réalisés dans un ordre précis. Voici le graphe de dépendance des Tps :



Dans ce cahier, les Tps sont donnés dans un ordre qui respecte les dépendances mais aussi le rythme du cours et la difficulté. Il est conseillé de faire les Tps dans l'ordre du cahier de TP.

Pour réaliser les Tps, vous disposez d'une archive avec beaucoup de code déjà fourni. L'archive est disponible sur la plateforme enseignement.

- Chaque répertoire correspond à un TP à réaliser (excepté le répertoire *utils* qu'il n'est pas nécessaire de modifier).
- Chaque répertoire contient un makefile qui permet de compiler tous les fichiers nécessaires (*make all*). Vous pouvez aussi nettoyer chaque répertoire à l'aide de la commande *make clean*.
- Chaque répertoire contient un fichier d'en-tête *.h* qui déclare les structures de données et les fonctions nécessaires à chaque TP. **Il est demandé de ne pas les modifier.**
- Chaque répertoire contient un programme de tests, souvent très basique et pas du tout exhaustif, pour réaliser les premiers tests sur les structures. Vous pouvez les modifier ou les enrichir comme vous le souhaitez.
- Enfin, chaque répertoire contient un fichier source *.c* qu'il convient de modifier/compléter pour réaliser le TP.

**Aucun rendu ne vous est demandé. Seul un examen de TP sera réalisé lors de la dernière séance, sans aucun document autorisé, ni accès à internet. Pendant cet examen, deux fonctions présentes dans le cahier de TP vous seront demandées. Une troisième fonction visant à résoudre un problème algorithmique sera aussi présente. Seules les fonctions marquées par [+] (jugées faciles) et [++] (jugées d'un niveau supérieure) sont attendues pour l'examen de TP. Les fonctions non marquées ne seront pas demandées en examen de TP.**

# TP – Listes

Temps prévu pour ce TP : 2h.

Dans ce TP, vous implémenterez le type abstrait liste : `List`. Les listes forment une structure de donnée de base pouvant servir à implémenter des piles, des files, des graphes ou encore des tables de hachage. Le fonctionnement des listes a été vues

## Implémentation des listes

Pour les piles, files ou graphes, nous avons besoin de listes capables de stocker des valeurs entières. Pour les tables de hachage, les listes doivent stocker des couples (`clé`, `valeur`) où la `clé` est une chaîne de caractères et `valeur` est un entier. Pour couvrir les deux situations, nous utiliserons des listes de couples (`clé`, `valeur`) où la `clé` sera le pointeur `NULL` si seules les valeurs nous intéressent.

Une liste est définie comme un pointeur sur la première cellule de la liste ou le pointeur `NULL` si la liste est vide. Une cellule sera composée d'une clé (chaîne de caractères ou `string`), d'une valeur (un entier) et d'un pointeur vers la cellule suivante. Ces structures sont définies en C de la manière suivante :

```
typedef char *string;

typedef struct cell{
    string key; /**< Key of the cell */
    int value; /**< Value of the cell */
    struct cell *nextCell; /**< Pointer to the next cell in the list */
} Cell, *List;
```

Il y a 3 fichiers fournis pour implémenter les listes : le fichier `list.h` contient toutes les déclarations nécessaires pour les listes (structures de cellule, de liste et fonctions associées) et il n'est pas nécessaire de le modifier. Le fichier `list.c` contient l'implémentation des fonctions associées aux listes. C'est ce fichier qu'il convient de modifier. Le fichier `testList.c` contient un programme pour tester les fonctions sur les listes. Vous êtes libres de le modifier comme vous l'entendez.

**Travail à faire :** implémentez les 6 fonctions suivantes sur les listes.

- `List newList()` : retourne une liste vide ;
- `[+] void freeList(List L)` : libère la mémoire occupée par la liste `L` ;
- `void printList(List L, int type)` : affiche une liste dans la console sous la forme :  
[valeur 1, valeur 2,...,valeur k] ou [(clé 1, valeur 1), (clé 2, valeur 2), ..., (clé k, valeur k)] selon que `type` vaut 0 ou non.
- `[+] Cell* findKeyInList(List L, string key)` : retourne un pointeur sur la première cellule de la liste `L` contenant la clé `key` ; `NULL` est retourné si la clé n'est pas trouvée.
- `[+] List delKeyInList(List L, string key)` : supprime la première cellule de la liste `L` contenant la clé `key`. La liste retournée est la nouvelle liste obtenue. Si la clé n'est pas présente, la liste en entrée n'est pas modifiée.
- `[+] List addKeyValueInList(List L, string key, int value)` : ajoute le couple (`key`, `value`) à la liste `L`, en tête de liste.

Attention, ces fonctions doivent fonctionner même si le pointeur `key` vaut `NULL`.

## Manipulation des chaînes de caractères :

Pour manipuler les clés (chaînes de caractères), nous utiliserons la bibliothèque `string.h`. Le mot-clé `string` est identique à la déclaration `char*` (pointeur sur un tableau de caractères). Cela signifie qu'il faut allouer la mémoire avant de pouvoir stocker ou manipuler les chaînes de caractères. C'est l'erreur la plus classique lorsque l'on utilise les chaînes de caractères en C. Faites attention ! Rappelons aussi qu'une chaîne de caractères bien formée en C se termine par le caractère nul '`\0`'. Vous trouverez à l'adresse suivante des exemples et toutes les fonctions de la bibliothèque `string` : <https://koor.fr/C/cstring/cstring.wp>

```
Is the list NULL (0=NO, 1=YES)? 1
Print empty list:[]
[]
Test findKeyList with empty list:
Key two not found

Test delKeyInList with empty list:
Print empty list after deletion:[]
Test add function:
[1]
[(one,1)]
[2,1]
[(two,2),(one,1)]
[3,2,1]
[(three,3),(two,2),(one,1)]
[0,3,2,1]
[(NULL,0),(three,3),(two,2),(one,1)]
[4,0,3,2,1]
[(four,4),(NULL,0),(three,3),(two,2),(one,1)]
Found key two with value 2
Key twelve not found
Found key (null) with value 0
Test delete function:NULL: [(four,4),(three,3),(two,2),(one,1)]
Test delete function:two: [(four,4),(three,3),(one,1)]
five: [(four,4),(three,3),(one,1)]
four: [(three,3),(one,1)]
one: [(three,3)]
three: []
three: []
```

trace d'exécution

## TP – File de priorité et tas

Temps prévu pour ce TP : 2h.

L'objectif de ce TP est d'implémenter la structure de file de priorité à l'aide d'un tas binaire.

Comme pour les précédents TP, un `makefile` et des fichiers de tests naïfs sont fournis. Il suffit de faire `make all` puis `./testheap` pour tester vos développements. La commande `make clean` permet de nettoyer le répertoire.

### File de priorité

Une file de priorité est une structure de données permettant de stocker des éléments auxquels est associée une priorité. Trois opérations sont possibles sur une file de priorité :

- insérer un élément dans la file en respectant sa priorité ;
- retourner l'élément avec la plus faible priorité ;
- supprimer l'élément avec la plus faible priorité.

A ces trois opérations, nous allons en ajouter une quatrième utile à l'algorithme de Prim :

- modifier la priorité d'un élément.

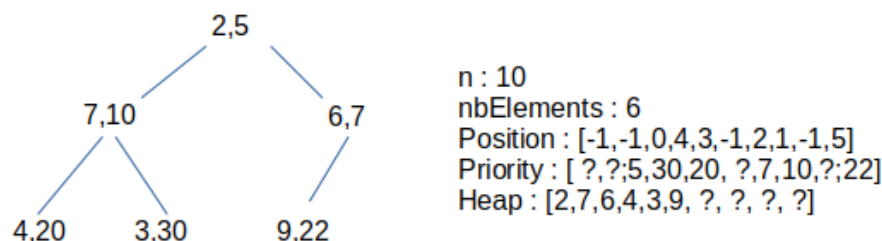
Cette dernière opération est contraignante car pour être efficace, il faut connaître à tout moment la position des éléments dans la file de priorité.

Une manière d'implémenter une file de priorité est d'utiliser une structure de tas (Heap). Les tas sont des arbres binaires complets tels que la priorité de tout nœud est inférieure ou égale à la priorité de ses descendants. Cette structure de tas vous a été présentée au premier semestre (peut-être en inversant la notion de priorité!). Elle peut être simplement mise en œuvre avec un tableau.

Dans les fichiers `heap.c` et `heap.h`, vous disposez de la déclaration d'une structure de tas.

```
typedef struct sheap{
    int n; //maximal number of elements
    int nbElements; //number of elements in the heap
    int *position; //array of the positions of the elements in the heap
    int *heap; //array used to implement the heap
    double *priority; //priority of each element used to insert the elements in the heap
} Heap;
```

Le champs `n` indique que les clés possibles sont  $\{0, \dots, n-1\}$ . Le nombre d'éléments dans le tas est donné par le champs `nbElements`. En particulier, toutes les clés ne sont pas nécessairement présentes dans le tas (`nbElements < n`). La priorité de la clé `k` est donnée par `priority[k]` et sa position dans le tas est donnée par `position[k]`. La position d'une clé non présente dans le tas sera fixée à `-1`. La figure suivante représente un tas possible sur les clés  $\{0, \dots, 9\}$  où l'élément prioritaire est celui de plus faible priorité:



Les points d'interrogation indiquent que la valeur présente dans les tables n'a pas d'importance (inutile de faire apparaître des '?' lors des affichages). La clé 2 est de priorité 5 qui est la plus faible de toutes les priorités des autres clés présentes.

**Travail à faire :** Il est demandé d'implémenter une file de priorité où la clé prioritaire est celle dont la priorité à la plus faible valeur. Pour cela, implémentez les fonctions suivantes :

- `Heap* createHeap(int n)` : crée un tas initialement vide dont les clés seront dans  $\{1, \dots, n\}$ .
- `void printHeap(Heap h)` : affiche en console les données du tas passé en paramètre. L'affichage pourra ressembler à celui des figures ci-dessus.
- `int getElement(Heap h)` : retourne l'élément de plus faible priorité dans le tas (dans notre cas, celui dont la valeur dans le tableau `priority` est la plus faible et qui se situe à la racine du tas).
- `[+] void insertHeap(Heap *h, int element, double priority)` : insère l'élément `element` dans le tas `h` avec la priorité `priority`.
- `[++] void modifyPriorityHeap(Heap *h, int element, double priority)` : modifie la priorité de l'élément `element` dans le tas `h` avec la nouvelle priorité `priority`. Attention, le tas peut nécessiter une restructuration.
- `[++] int removeElement(Heap *h)` : supprime l'élément de plus faible priorité dans le tas (dans notre cas, celui dont la valeur dans le tableau `priority` est la plus faible et qui se situe à la racine du tas) et retourne cet élément.

Une fonction `swap` est fournie pour échanger la position de deux éléments dans le tas.

```
Current heap:
n: 5
nbElements: 0
position: [-1 -1 -1 -1 -1 ]
priority: [-1.00 -1.00 -1.00 -1.00 -1.00 ]
heap: []
Current heap:
n: 5
nbElements: 1
position: [-1 0 -1 -1 -1 ]
priority: [-1.00 5.00 -1.00 -1.00 -1.00 ]
heap: [1 ]
Current heap:
n: 5
nbElements: 2
position: [-1 1 0 -1 -1 ]
priority: [-1.00 5.00 2.00 -1.00 -1.00 ]
heap: [2 1 ]
Current heap:
n: 5
nbElements: 3
position: [-1 1 0 2 -1 ]
priority: [-1.00 5.00 2.00 4.00 -1.00 ]
heap: [2 1 3 ]
Current heap:
n: 5
nbElements: 4
position: [-1 3 1 2 0 ]
priority: [-1.00 5.00 2.00 4.00 1.00 ]
heap: [4 2 3 1 ]
Current heap:
n: 5
nbElements: 5
position: [4 3 1 2 0 ]
priority: [3.00 5.00 2.00 4.00 1.00 ]
heap: [4 2 3 1 0 ]
Element with lowest priority: 4
Heap after modifying priority of element 4:
n: 5
nbElements: 5
position: [1 3 0 2 4 ]
priority: [3.00 5.00 2.00 4.00 3.50 ]
heap: [2 0 3 1 4 ]
Heap after removing element with lowest priority:
n: 5
nbElements: 4
position: [0 3 -1 2 1 ]
priority: [3.00 5.00 2.00 4.00 3.50 ]
heap: [0 4 3 1 ]
Removed element: 2
```

Voici une trace d'exécution avec le fichier test fourni.

## TP – Tables de hachage

Temps prévu pour ce TP : 4h.

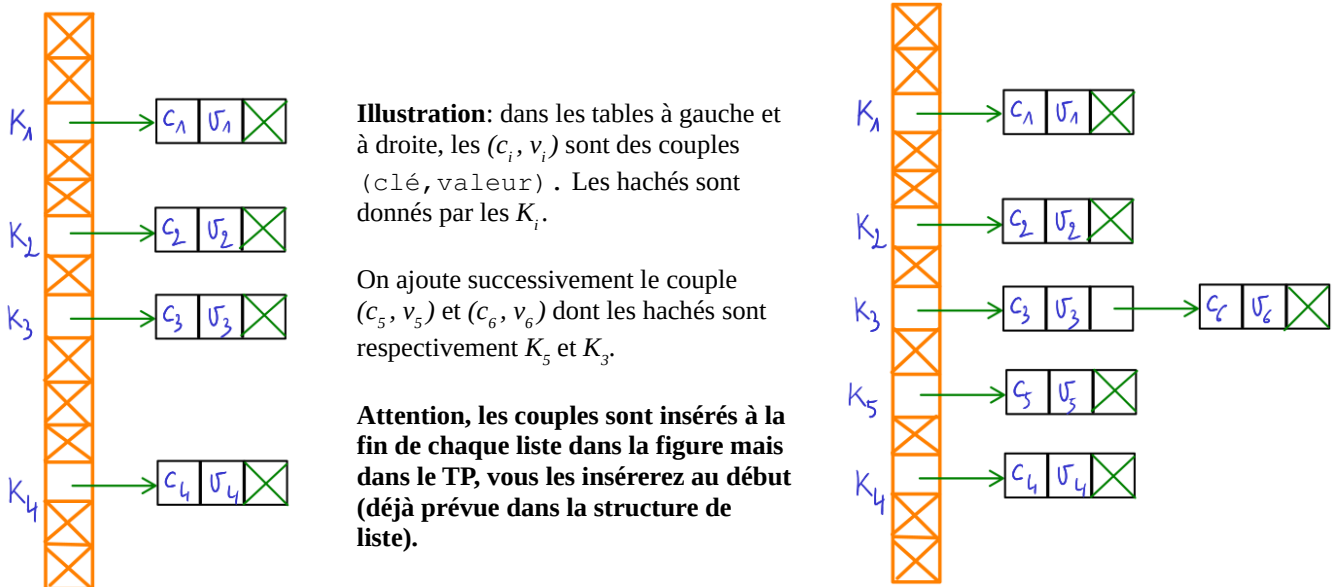
L'objectif est d'implémenter une structure de table de hachage s'appuyant sur la structure de liste des précédents TPs.

### Rappel sur les principes d'une table de hachage :

Les tables de hachage sont des structures de données qui permettent de stocker des couples (clé, valeur) de manière efficace. Les clés et les valeurs peuvent être des entiers, des chaînes de caractères ou des objets beaucoup plus complexes. L'objectif est de pouvoir rechercher, insérer ou supprimer en temps constant des couples mais en pratique, cette complexité sera atteinte uniquement en moyenne (ou en complexité amortie).

Dans ce TP, une table de hachage sera un tableau de listes, chaque liste pouvant stocker des couples (clé, valeur). Les clés seront des chaînes de caractères et les valeurs des entiers. Une fonction de hachage, qui transforme des clés en entiers, est aussi nécessaire.

Pour ajouter un couple (clé, valeur) dans la table, on commence par calculer le haché de la clé,  $\text{hash}(\text{clé})$ , qui retourne un indice dans le tableau de listes. Ensuite, on ajoute le couple (clé, valeur) dans la liste située à la position  $\text{hash}(\text{clé})$  calculée précédemment.



**Fonction de hachage :** la fonction de hachage a un rôle très important car elle doit assurer une bonne répartition dans le tableau et éviter le plus possible les collisions (deux clés différentes ayant le même haché). Dans ce TP, nous utiliserons la fonction de hachage non cryptographique, *MurmurHash* dont une description précise peut être trouvée par exemple sur wikipedia : <https://en.wikipedia.org/wiki/MurmurHash>.

Cette fonction est simple, rapide et offre de bonnes propriétés statistiques. Une implémentation en C de cette fonction est donnée sur la plateforme via les fichiers fournis pour les tables de hachage. Pour obtenir le haché d'une chaîne de caractères, il suffit d'appeler la fonction `murmurhash` :

```
size_t murmurhash(string key, size_t sizeKey, size_t maxValue) ;
```

Cette fonction retourne un entier compris entre 0 (inclus) et `maxValue` (exclus) qui est le hash de la clé `key`. Le nombre de caractères de la clé est donné par l'entier `sizeKey`.

**Gestion du remplissage :** plus une table de hachage est remplie, moins elle devient efficace car la taille des listes a tendance à augmenter. En pratique si le nombre d'éléments dans la table dépasse un certain seuil, la taille du tableau de listes est doublée. Il faut alors insérer un à un tous les éléments de l'ancienne table dans la nouvelle. C'est une opération coûteuse mais qui heureusement, arrive très rarement.

Dans ce TP, le redimensionnement du tableau de listes se fera lorsque le nombre d'éléments dans la table dépasse la taille du tableau. Cela signifie qu'en moyenne (seulement), chaque liste contiendra au plus un couple (clé,valeur).

## Travail à réaliser

Un fichier de tests est donné qui permet de tester toutes les fonctions sur les tables de hachage (fichier *testhashtable.c*). Chacune des fonctions a une fonction de test spécifique. Il suffit donc de commenter ou non les lignes correspondantes.

Dans le fichier *hashtable.h*, la structure de données *HashTable* est définie comme suit :

```
typedef struct hashtable{
    size_t sizeTable;
    size_t numberOfPairs;
    List *table;
} HashTable;
```

Les champs correspondent à la taille du tableau de listes (*sizeTable*), les tableau de listes (*table*) et le nombre de couples (clé,valeur) dans la table (*numberOfPairs*). La fonction de hachage *murmurhash* est également donnée ainsi que la fonction d'affichage d'une table (*hashtablePrint*). Toutes les autres fonctions sont à implémenter entièrement sauf la fonction *countDistinctWordsInBook* qu'il suffit de modifier.

Voici les fonctions à implémenter (de préférence, dans cet ordre) :

- *HashTable hashtableCreate(size\_t sizeTable)* : retourne une table vide (sans élément) mais dont le tableau de listes peut contenir *sizeTable* listes. Au départ, ces listes sont vides.
- *[+] void hashtableInsertWithoutResizing(HashTable \*hashtable, string key, int value)* : Insère dans la table pointée par *hashtable* le couple (*key,value*). Si la clé est déjà présente, la nouvelle valeur *value* doit remplacer l'ancienne. Lors de cette insertion, la taille du tableau de listes ne doit pas être modifiée.
- *[+] void hashtableDestroy(HashTable \*hashtable)* : libère la mémoire occupée par les champs de la table pointée par *hashtable*. Les champs *sizeTable* et *numberOfPairs* sont mis à 0. La mémoire occupée par le tableau de listes est libérée et le tableau de listes est mis à *NULL*.
- *[++] HashTable hashtableDoubleSize(HashTable hashtable)* : fonction qui retourne une table de hachage contenant tous les couples (clé,valeur) de la table *hashtable* mais dont la capacité du tableau de listes est doublée. La table *hashtable* ne doit pas être modifiée.
- *[++] void hashtableInsert(HashTable \*hashtable, string key, int value)* : insère le couple (*key,value*) dans la table pointée par *hashtable*. Si nécessaire, la capacité de la table doit être augmentée.
- *[+] int hashtableHasKey(HashTable hashtable, string key)* : fonction qui teste si la clé *key* est dans la table de hachage *hashtable*. Elle retourne 1 si c'est le cas, 0 sinon.
- *[+] int hashtableGetValue(HashTable hashtable, string key)* : fonction qui retourne la valeur associée à la clé *key* dans la table *hashtable*. La clé est supposée être dans la table.
- *[+] int hashtableRemove(HashTable \*hashtable, string key)* : fonction qui supprime de la table pointée par *hashtable*, le couple associée à la clé *key*. La fonction retourne 1 si la suppression a eu lieu (clé trouvée) et 0 sinon.
- *void countDistinctWordsInBook()* : fonction qui affiche le nombre total de mots et le nombre total de mots distincts dans le fichier « potter-clean.txt ». Le code fourni permet de lire chaque mot du fichier et de l'afficher. Modifier le pour avoir l'affichage attendu.

## TP – File

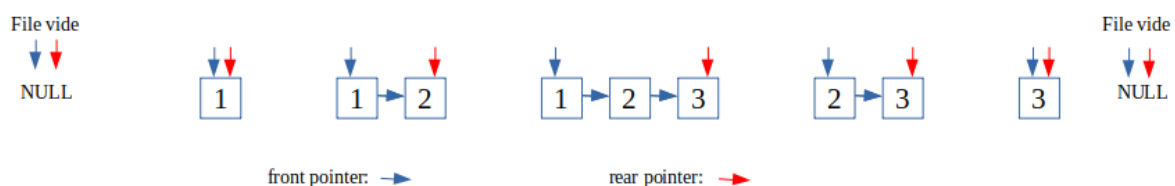
Temps prévu pour ce TP : 2h.

### Implémentation de la structure de file

Une file est une structure de donnée de type FIFO : first in, first out. Autrement dit, le premier élément à entrer dans la file sera le premier élément à en sortir. Une file peut être représentée par une structure de liste à laquelle on ajoute un pointeur vers la queue de la liste. Ainsi, on accède en temps constant à la tête et à la queue de la liste.

Dans ce TP, chaque élément ajouté le sera à la queue de la liste. En revanche, chaque élément supprimé le sera en tête de liste.

Voici un exemple d'exécution qui ajoute les 3 valeurs 1, 2 et 3 puis qui les supprime ensuite :



Trois fichiers sont fournis pour déclarer (`queue.h`), implémenter (`queue.c`) et tester (`testqueue.c`) une structure de file basée sur une liste simplement chaînée. Un `makefile` est aussi fourni pour tout compiler ou nettoyer (`make all` ou `make clean`).

Dans ce TP, une file est une structure composée de deux champs : le devant (`front`) et l'arrière (`rear`) de la liste qui sont des pointeurs sur le premier et le dernier nœuds de la liste.

```
typedef struct queue {
    Cell* front; /** A pointer to the front (first) element in the queue. */
    Cell* rear; /** A pointer to the rear (last) element in the queue. */
} Queue;
```

Il faut donc avoir implémenté les fonctions sur listes avant de réaliser ce TP.

- Implémentez les fonctions `createQueue`, `queuePrint`, `[+] enqueue`, `[+] dequeue`, `isEmpty` et `[+]vqueueGetFrontValue` dans le fichier `queue.c`. Les noms sont suffisamment explicites ainsi que les commentaires pour ne pas décrire ces fonctions ici.

Un exemple d'exécution du programme de test est donné ci-contre.

```
[ ]
Add 1 to queue:
[1]
Add 2 to queue:
[1,2]
Add 3 to queue:
[1,2,3]
Dequeued value: 1
Updated queue:
[2,3]
Dequeued value: 2
Updated queue:
[3]
Front value: 3
add 4 to queue:
[3,4]
Is queue empty? 0
Final queue:
[ ]
Is queue empty? 1
```

## TP – Pile

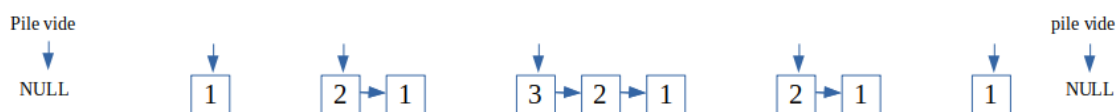
Temps prévu pour ce TP : 2h.

### Implémentation de la structure de Pile

Une pile (*stack*) est une structure de donnée de type LIFO : last in, first out. Autrement dit, le dernier élément à entrer dans la pile sera le premier élément à en sortir (pensez à une pile d'assiettes). Une pile peut tout simplement être représentée par une structure de liste. Ainsi, on accède en temps constant à la tête de la liste.

Dans ce TP, chaque élément ajouté ou supprimé le sera en tête de la liste.

Voici un exemple d'exécution qui ajoute les 3 valeurs 1, 2 et 3 puis qui les supprime ensuite :



Trois fichiers sont fournis pour déclarer (*stack.h*), implémenter (*stack.c*) et tester (*teststack.c*) une structure de pile basée sur une liste simplement chaînée. Un *makefile* est aussi fourni pour tout compiler ou nettoyer (*make all* ou *make clean*).

Dans ce TP, la structure de pile est identique à la structure de liste dans laquelle tous les champs *key* de toutes les cellules seront à NULL. Cela donne donc la déclaration suivante :

```
typedef List Stack;
```

Il faut évidemment avoir implémenté les fonctions sur listes avant de réaliser ce TP.

- Implémentez les fonctions *createStack*, *[+] push*, *[+] pop*, *[+] peek*, *isStackEmpty* et *stackPrint* dans le fichier *stack.c*. Les noms sont suffisamment explicites ainsi que les commentaires pour ne pas décrire ces fonctions ici.

Un exemple d'exécution du programme de test est donné ci-contre.

```
[ ]
Add 1 to stack:
[1]
Add 2 to stack:
[2,1]
Add 3 to stack:
[3,2,1]
popped value: 3
Updated stack:
[2,1]
popped value: 2
Updated stack:
[1]
add 4 to stack:
[4,1]
Top value of the stack: 4
[4,1]
Is stack empty? 0

Final stack:
[ ]
Is stack empty? 1
```

## TP – Arbres binaires de recherche

Temps prévu pour ce TP : 2h.

L'objectif de ce TP est d'implémenter le type abstrait vu en cours `BinarySearchTree` pour les arbres binaires de recherche. Nous nous limiterons au cas où les valeurs stockées sont des entiers.

**Fichiers fournis :** pour ce TP, vous utiliserez les fonctions fournies par les fichiers `utils.h` et `utils.c` dans le répertoire `utils`. La structure de données `BinarySearchTree` ainsi que toutes les fonctions nécessaires à la réalisation du TP sont déclarées dans le fichier `bst.h` du répertoire `bst`.

Voici la liste des fichiers fournis avec quelques informations :

- `utils.h` et `utils.c` : `utils.c` implémente 3 fonctions mais seulement 2 sont utiles pour ce TP. La fonction `uniformRandomPermutation` génère une permutation aléatoire uniforme, stockée dans un tableau passé en paramètre, la taille du tableau étant aussi fournie. À l'inverse, `biasedRandomPermutation` génère une permutation aléatoire non uniforme avec une longue sous-suite triée au début du tableau (les 10 premiers%). Ces deux fichiers n'ont pas besoin d'être modifiés et ne seront utiles que pour faire des tests.
- `bst.c` et `bst.h` : ces deux fichiers déclarent et implémentent toutes les fonctions sur les arbres binaires de recherche. En particulier, la structure `BinarySearchTree` est déclarée dans le `.h` et correspond à celle vue en cours (pointeur sur un nœud de type `NodeBST`). Une partie du TP consiste à compléter le fichier `.c` où certaines fonctions n'ont pas été implémentées. Le fichier `.h` n'a pas besoin d'être modifié.
- `testbst.c` : ce fichier contient le `main` qui appelle 3 fonctions de tests : `testBst`, `compare_bst` et `draw_graph_bst`. `TestBst` implémente quelques tests basiques des fonctions sur les BST. La fonction `compare_bst` compare le comportement moyen de 3 paramètres (le temps de construction, la hauteur et le temps de recherche) et fait un affichage console. La fonction `draw_graph_bst` sauvegarde dans le fichier `data.gnuplot` les données pour les 3 paramètres d'intérêt pour différentes tailles. Ce fichier peut ensuite être utilisé pour tracer des courbes avec `gnuplot` et le script `gnuplot graph.gnuplot`. Vous êtes libres de modifier le fichier `testbst.c` comme vous l'entendez. Néanmoins, commenter/dé-commenter des parties de code devrait suffire dans un premier temps. La fonction `draw_graph_bst` peut être un peu longue. Il peut être intéressant de la commenter dans un premier temps.
- `Makefile` : permet de compiler tous les fichiers. Normalement, il ne sera pas nécessaire de le modifier.

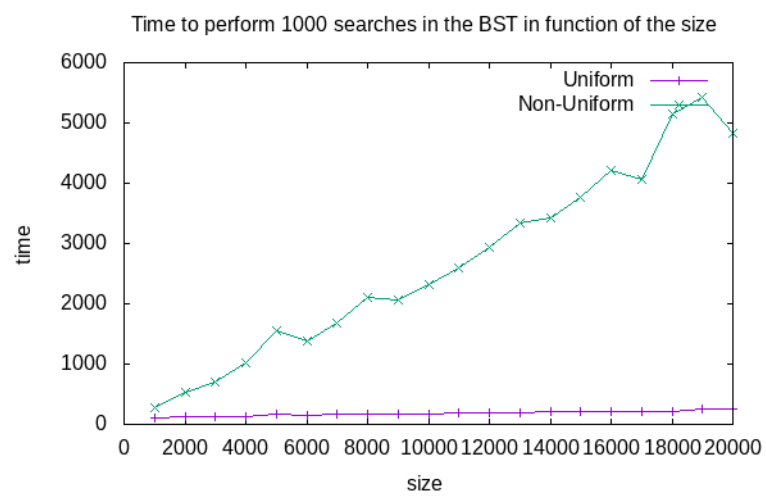
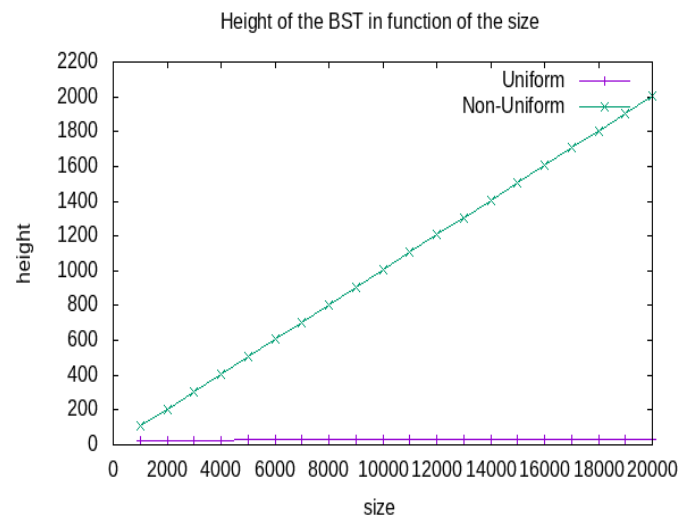
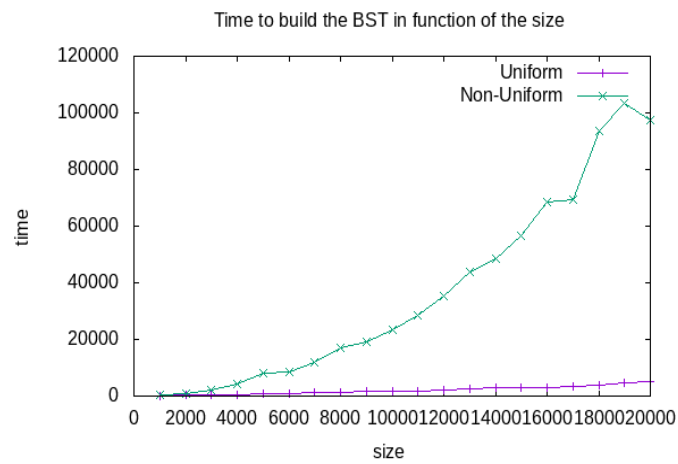
### Implémentation de la structure d'arbre binaire de recherche

Toutes les fonctions étant bien nommées et commentées, il est facile de deviner leur objectif. Implémenter les fonctions suivantes :

- `createEmptyBST` : retourne un arbre vide ;
- `[+] freeBST` : libère la mémoire occupée par un BST passé en paramètre ;
- `[+] addToBST` : ajoute une valeur dans le BST passé en paramètre ;
- `heightBST` : calcule la hauteur de l'arbre passé en paramètre ;
- `[+] searchBST` : recherche une valeur dans un BST ;
- `buildBSTFromPermutation` : construit un BST en ajoutant un à un les éléments de la permutation.

La fonction `prettyPrintBST`, qui permet un affichage correct des « petits arbres », est donnée.

Quelles conclusions pouvez-vous tirer des résultats d'expériences que vous obtenez ? Les fonctions de suppressions (`deleteRootBST` et `deleteFromBST`) sont optionnelles.



## TP – Arbres binaires de recherche randomisés

Temps prévu pour ce TP : 3h

L'objectif de ce TP est d'implémenter le type abstrait vu en cours `RbinarySearchTree` pour les arbres binaires de recherche randomisés. Nous nous limiterons une fois de plus au cas où les valeurs stockées sont des entiers. En plus de l'implémentation des fonctions, il sera demandé de comparer la complexité de quelques unes de ces fonctions avec les mêmes fonctions pour les ABR (cf précédent TP, à faire avant).

### Fichiers fournis

**Fichiers fournis :** pour ce TP, vous utiliserez les fonctions fournies par les fichiers `utils.h` et `utils.c` dans le répertoire `utils`. La structure de données `RBinarySearchTree` ainsi que toutes les fonctions nécessaires à la réalisation du TP sont déclarées dans le fichier `rbst.h` du répertoire `rbst`.

Voici la liste des fichiers fournis avec quelques informations :

- `utils.h` et `utils.c` : `utils.c` implémente 3 fonctions mais seulement 2 sont utiles pour ce TP. La fonction `uniformRandomPermutation` génère une permutation aléatoire uniforme, stockée dans un tableau passé en paramètre, la taille du tableau étant aussi fournie. A l'inverse, `biasedRandomPermutation` génère une permutation aléatoire non uniforme avec une longue sous-suite triée au début du tableau (les 10 premiers %). Ces deux fichiers n'ont pas besoin d'être modifiés et ne seront utiles que pour faire des tests.
- `rbst.c` et `rbst.h` : ces deux fichiers déclarent et implémentent toutes les fonctions sur les arbres binaires de recherche randomisés. En particulier, la structure `RBinarySearchTree` est déclarée dans le `.h` et correspond à celle vue en cours (pointeur sur un nœud de type `NodeRBST`). Une partie du TP consiste à compléter le fichier `.c` où certaines fonctions n'ont pas été implémentées. Le fichier `.h` n'a pas besoin d'être modifié.
- `test_rbst.c` : ce fichier contient le main qui appelle 3 fonctions de tests : `testRBst`, `compare_bst_rbst` et `draw_graph_rbst`. `TestRBst`. La fonction `testRBst` implémente quelques tests basiques des fonctions sur les BST randomisés. La fonction `compare_bst_rbst` compare le comportement moyen de 3 paramètres (le temps de construction, la hauteur et le temps de recherche) pour les BST et rBST et fait un affichage console. La fonction `draw_graph_rbst` sauvegarde dans le fichier `data.gnuplot` les données pour les 3 paramètres d'intérêt pour différentes tailles et pour les deux types d'arbres. Ce fichier peut ensuite être utilisé pour tracer des courbes avec `gnuplot` et le script `gnuplot graph.gnuplot`. Vous êtes libres de modifier le fichier `test_rbst.c` comme vous l'entendez. Néanmoins, commenter/dé-commenter des parties de code devrait suffire dans un premier temps. La fonction `draw_graph_rbst` peut être un peu longue. Il peut être intéressant de la commenter dans un premier temps.
- `Makefile` : permet de compiler tous les fichiers. Normalement, il ne sera pas nécessaire de le modifier. Notez que `make graph` suffit pour générer le graphique à partir du fichier de données `data.gnuplot`.

### Travail à réaliser sur les ABR randomisés (RBST)

Toutes les fonctions étant bien nommées et commentées, il est facile de deviner leur objectif. Implémenter les fonctions suivantes :

- `createEmptyRBST` : retourne un arbre vide ;
- `[+] freeRBST` : libère la mémoire occupée par un RBST passé en paramètre ;
- `[+] sizeofRBST` : retourne le nombre de nœuds de l'arbre.
- `[++] addToRBST` : ajoute une valeur dans le RBST passé en paramètre ;

- `heightRBST` : calcule la hauteur de l'arbre passé en paramètre ;
- `[+] searchRBST` : recherche une valeur dans un RBST et retourne un pointeur sur le nœud correspondant ;
- `buildRBSTFromPermutation` : construit un RBST en ajoutant un à un les éléments de la permutation.

La fonction `prettyPrint RBST`, qui permet un affichage correct des « petits arbres », est donnée.

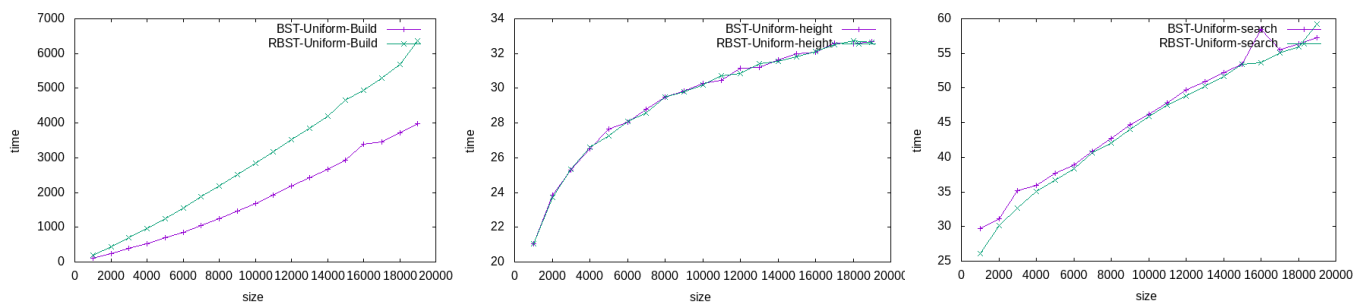
Attention, la fonction `addToRBST` nécessite deux fonctions auxiliaires qu'il faut implémenter :

- `[+] insertAtRoot` : qui insère une valeur à la racine de l'arbre en coupant l'arbre selon la valeur (fonction `Split` du cours) puis en recollant les morceaux ;
- `[++] splitRBST` : (fonction `Split` du cours) fonction qui construit deux sous-arbres INF et SUP qui contiennent respectivement les nœuds du RBST plus petits, respectivement plus grands, que la valeur passée en paramètre. Les arbres INF et SUP sont retournés grâce à un passage de paramètre par pointeur.

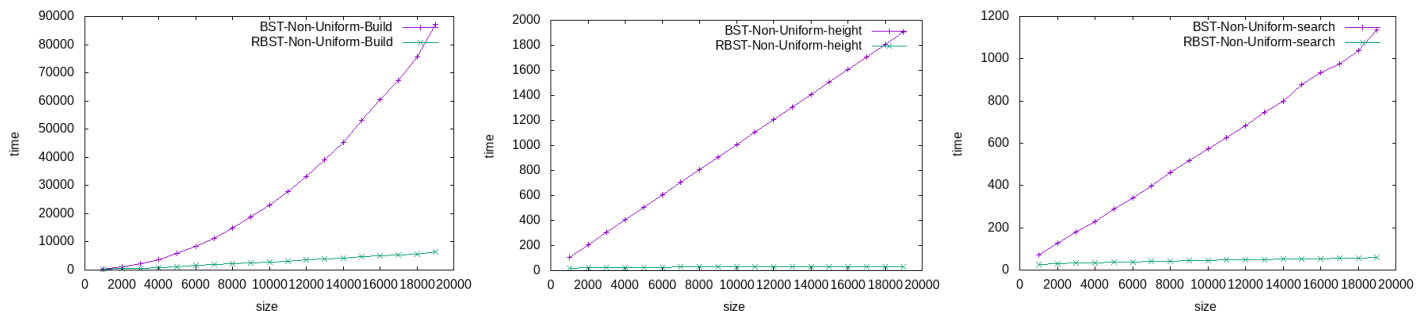
Quelles conclusions pouvez-vous tirer des résultats d'expériences que vous obtenez ?

Voici des exemples de graphiques dessinés avec gnuplot :

### Cas uniforme



### Cas non-uniforme



## TP – Arbres rouges-noirs (optionnel)

Temps prévu pour ce TP : 1 séance.

L'objectif de ce TP est d'implémenter le type abstrait `RedBlackBST` pour les arbres binaires de recherche rouges-noirs.

Comme pour le précédent TP, nous nous limiterons au cas où les valeurs stockées sont des entiers. En plus de l'implémentation des fonctions associées à ce type abstrait, il sera demandé de le comparer aux types abstraits du premier TP (ABR et ABR randomisé). Pour cela, toutes les fonctions de tests sont prêtes, y compris celles pour tracer les graphiques.

### Fichiers fournis

Voici la liste des fichiers nécessaires avec quelques informations :

- `utils.h` et `utils.c` : voir Tps précédents.
- `bst.c` et `bst.h` : voir Tps précédents.
- `rbst.c` et `rbst.h` : voir Tps précédents.
- `test_rbbst.c` : ce fichier contient le `main` qui appelle 3 fonctions de tests : `testRedBlackBST`, `compare_data_structures` et `draw_graph_trees`. La fonction `testRedBlackBST` implémente quelques tests basiques des fonctions sur les arbres rouges-noirs. La fonction `compare_data_structures` compare le comportement moyen de 3 paramètres (le temps de construction, la hauteur et le temps de recherche) pour les BST, RBST et `RedBlackBST` et fait un affichage console. La fonction `draw_graph_trees` sauvegarde dans le fichier `data.gnuplot` les données pour les 3 paramètres d'intérêt pour différentes tailles et pour les troistypes d'arbres. Ce fichier peut ensuite être utilisé pour tracer des courbes avec `gnuplot` et le script `gnuplot graph.gnuplot`. Vous êtes libres de modifier le fichier `test_rbbst.c` comme vous l'entendez. Néanmoins, commenter/dé-commenter des parties de code devrait suffire dans un premier temps. La fonction `draw_graph_trees` peut être un peu longue. Il peut être intéressant de la commenter dans un premier temps.
- `Makefile` : permet de compiler tous les fichiers. Normalement, il ne sera pas nécessaire de le modifier. Notez que `make graph` suffit pour générer le graphique à partir du fichier de données `data.gnuplot`.

### Code source déjà fourni sur les arbres rouges-noirs

Beaucoup de fonctions spécifiques aux arbres rouges-noirs sont déjà implémentées pour faciliter le TP :

- la création d'un arbre vide via la fonction `createEmptyRedBlackBST` ;
- les rotations gauche (`leftRotationRedBlackBST`) et droite (`rightRotationRedBlackBST`) ;
- le calcul de la hauteur d'un arbre rouge-noir (`heightRedBlackBST`) ;
- la recherche d'un élément dans l'arbre (`searchRedBlackBST`) ;
- le calcul de la hauteur noire d'un arbre (`blackHeightRedBlackBST`). Attention, cette fonction retourne la hauteur noire +1 si la racine est noire et elle retourne -1 si elle détecte une erreur dans la hauteur noire des sous arbres (qui doivent être égales!) ;
- La création d'un arbre rouge-noir à partir d'une permutation (`buildRedBlackBSTFromPermutation`) ;
- L'affichage d'un arbre rouge-noir (`prettyPrintRedBlackBST`).

### Travail à réaliser sur les arbres rouges-noirs (RedBlackBST)

Implémentez les fonctions suivantes :

- `freeRedBlackBST` : libère la mémoire occupée par un arbre rouge-noir passé en paramètre ;
- `balanceRedBlackBST(RedBlackBST *tree, NodeRedBlackBST *curr)` : l'ajout d'une valeur dans un arbre rouge-noir se fait en deux temps. Tout d'abord, la valeur est insérée à une feuille comme dans un arbre binaire de recherche classique, la couleur du nouveau nœud étant fixée à rouge. Ensuite, il faut réparer l'arbre car il peut y avoir deux nœuds rouges successifs ou bien une racine rouge. Pour cela, on utilise les rotations (voir cours). La fonction `balanceRedBlackBST` implémente cette deuxième phase. L'arbre est passé en paramètre mais par pointeur car il peut être modifié (la racine peut changer). Le nœud qu'il faut traiter est aussi passé en paramètre (au départ, il s'agit du nœud de la nouvelle valeur insérée).
- `insertNodeRedBlackBST` : cette fonction implémente l'ajout d'une valeur dans un arbre rouge-noir passé en paramètre. Comme l'insertion peut changer la racine, l'arbre est passé en paramètre par pointeur.
- `isRedBlackBST` : teste si l'arbre passé en paramètre est bien un arbre rouge-noir. Vous pouvez (devez) utiliser la fonction qui calcule la hauteur noire.

Grâce aux fonctions de tests fournies et à la génération automatique des graphiques, faites une analyse critique des 3 types abstraits d'arbres binaires de recherche implémentés.

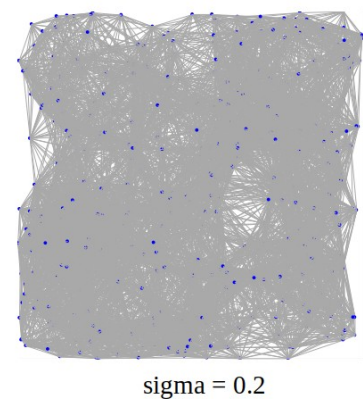
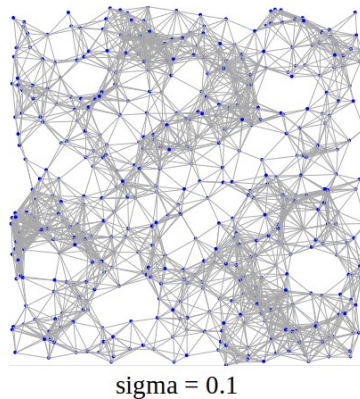
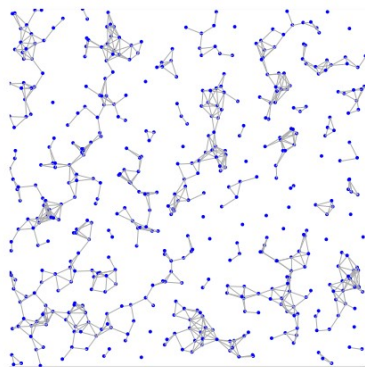
## TP – Parcours dans les graphes

Temps prévu pour ce TP : 4h.

L'objectif premier du TP est d'implémenter la structure de graphe, orienté ou non, à partir de la représentation par tableau de liste de successeurs. Dans un deuxième temps, les parcours en largeur et en profondeur seront implémentés.

Pour la mise en œuvre des listes de successeurs et des parcours, il faut tout d'abord implémenter les structures de liste (List), de file (Queue) et de pile (Stack) des précédents TPs.

Dans ce TP, les graphes seront construits à partir de points tirés au hasard dans  $[0,1] \times [0,1]$ . Les sommets du graphe seront les points tirés au hasard. Une arête non orientée  $\{A,B\}$  avec  $A=(x,y)$  et  $B=(x',y')$  sera présente dans le graphe si la distance entre A et B est plus petite qu'un seuil  $\sigma$  fixé dans  $[0,1]$ . Voici trois exemples de graphes non orientés ayant 500 sommets obtenus avec différentes valeurs de sigma.



Si le graphe est orienté, une arête  $(A,B)$  (de A vers B) avec  $A=(x,y)$  et  $B=(x',y')$  sera présente dans le graphe si la distance entre A et B est plus petite qu'un seuil  $\sigma$  fixé dans  $[0,1]$  et si  $y < y'$ . Dans les graphiques précédents, cela revient à orienter les arêtes du haut de l'image vers le bas. **Dans ce TP dédié essentiellement aux graphes non orientés, aucun test sur les graphes orientés n'ont été implémentés.**

### Génération et représentation des graphes par tableau de listes d'adjacence

Conserver les coordonnées des sommets et le sigma suffit pour réaliser tous les algorithmes car cela revient à manipuler la matrice d'adjacence (tester si une arête existe entre deux sommets se fait par un calcul simple de distance).

Néanmoins, les algorithmes de parcours sont plus efficaces avec la représentation par tableau de listes d'adjacence, surtout si le graphe est peu dense.

La structure de graphe est définie comme suit dans le fichier `graph.h` avec une structure de liste classique de type List (voir précédents Tps, à faire avant).

```
typedef struct Graph {
    int numberVertices; /** The number of vertices in the graph. */
    List *array; /** Array of adjacency lists. A null pointer means an empty list */

    double *xCoordinates; /** array of the x coordinates of the points of each vertex */
    double *yCoordinates; /** array of the y coordinates of the points of each vertex */
    double sigma; /** maximal distance between two neighboring vertices */

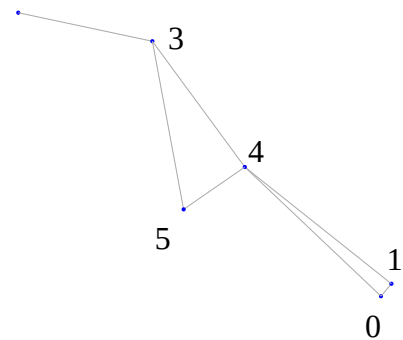
    int *parents; /** array to store the parent of each node for the covering tree. */

    int *topological_ordering; /** array to store a topological ordering of the graph. */
    double *earliest_start; /** array to store the earliest start of each node. */
    double *latest_start; /** array to store the latest start of each node. */
} Graph;
```

Le nombre de sommets est donné par le champ `numberVertices`. Les coordonnées des points sont stockées dans les deux tableaux `xCoordinates` et `yCoordinates`. Le champ `sigma` correspond au paramètre sigma. Le tableau `parents` servira uniquement à construire les arbres couvrants à partir des parcours en profondeur et largeur. Toutes ses valeurs seront initialisées à -1. Les 3 autres tableaux seront utilisés dans d'autres Tps sur les graphes.

Pour le graphe ci-contre ayant 6 sommets et généré avec  $\sigma = 0.5$ , nous obtenons la structure suivante :

```
numberVertices : 6
sigma = 0.5
xCoordinates = [0.91,0.93,0.04,0.36,0.58,0.44]
yCoordinates = [0.71,0.68,0.03,0.1,0.4,0.5]
array[0] : 4→1→NULL          array[4] : 5→3→1→0→NULL
array[1] : 4→0→NULL          array[5] : 4→3→NULL
array[2] : 3→NULL
array[3] : 5→4→2→NULL
parents=[-1,-1,-1,-1,-1,-1]
```



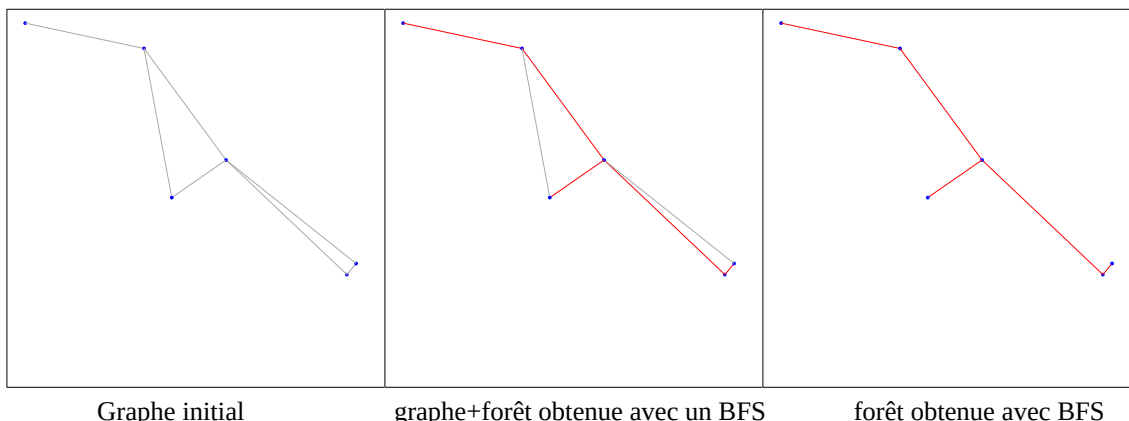
### Travail à réaliser :

- Implémentez les fonctions `[+] addEdgeInGraph`, `createGraph` et `printConsoleGraphe`. Les intitulés et les descriptions des fonctions sont suffisamment claires pour ne pas les décrire ici. L'affichage d'un graphe dans la console pourra ressembler à l'image ci-contre (attention, cela ne correspond pas au graphe précédent).

```
Print graph:
Vertex 0: (0.050319,0.219413)
Vertex 1: (0.807597,0.782765)
Vertex 2: (0.221314,0.461621)
Vertex 3: (0.436210,0.321816)
Vertex 4: (0.258311,0.330447)
Vertex 5: (0.600169,0.790168)
Vertex 0: 4 -> 2 -> NULL
Vertex 1: 5 -> NULL
Vertex 2: 4 -> 3 -> 0 -> NULL
Vertex 3: 4 -> 2 -> NULL
Vertex 4: 3 -> 2 -> 0 -> NULL
Vertex 5: 1 -> NULL
parents: -1 -1 3 4 0 -1
Topological ordering:
[-1 -1 -1 -1 -1 -1]
Print start dates:
vertex 0 : [earliest start date=-1.00, latest start date=-1.00 ]
vertex 1 : [earliest start date=-1.00, latest start date=-1.00 ]
vertex 2 : [earliest start date=-1.00, latest start date=-1.00 ]
vertex 3 : [earliest start date=-1.00, latest start date=-1.00 ]
vertex 4 : [earliest start date=-1.00, latest start date=-1.00 ]
vertex 5 : [earliest start date=-1.00, latest start date=-1.00 ]
```

Une fonction `drawGraph` est fournie qui permet de sauvegarder dans une image de type `svg` un graphe passé en paramètre. La fonction peut sauvegarder le graphe (paramètre `type=0`), la forêt couvrante stockée dans le tableau `parents` et calculée à partir de parcours (paramètre `type=1`), le graphe et la forêt couvrante superposés (paramètre `type=2`) et l'ordre topologique ainsi que les dates de départ des prochains Tps (`type=3`). Il est aussi possible d'indiquer si le graphe est orienté ou non.

Voici trois exemples d'images obtenues (`type=0,1,2`) pour des graphes non orientés.



Graphe initial

graphe+forêt obtenue avec un BFS

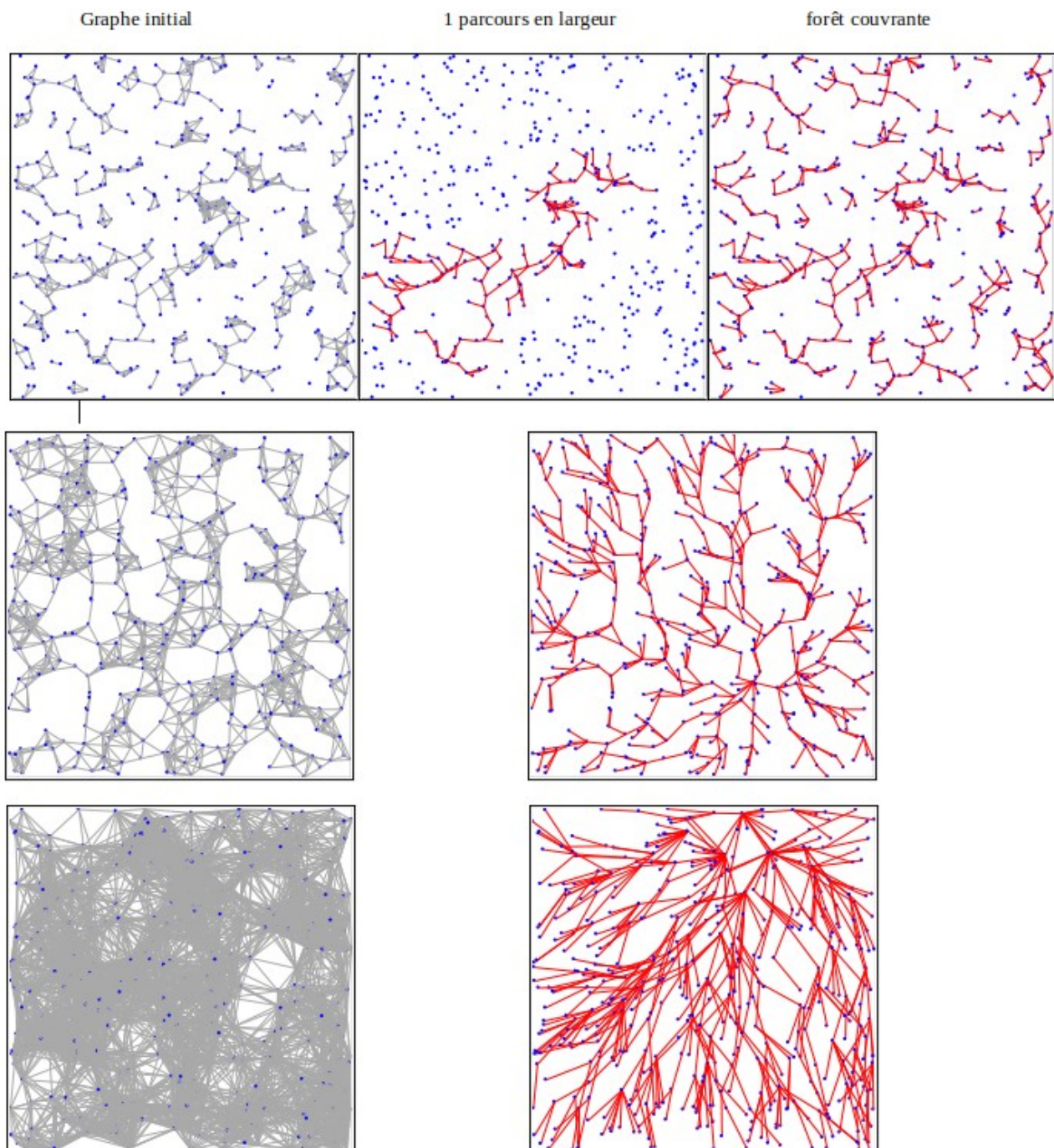
forêt obtenue avec BFS

Lorsque qu'une recherche en profondeur ou en largeur est réalisée à partir d'un sommet, le parcours va visiter tous les sommets de la composante connexe du sommet initial. Pour construire une forêt couvrante, il faut donc lancer autant de parcours qu'il n'y a de composantes connexes.

### Travail à réaliser :

- Implémentez les fonctions `[++] graphDFS` et `[++] graphBFS` qui réalisent respectivement un parcours en profondeur et en largeur sur un graphe donné à partir d'un sommet lui aussi donné. Stockez la structure sous-jacente de forêt couvrante dans le champs `parents`.
- Implémentez la fonction `[+] numberOfComponents` qui calcule le nombre de composantes connexes d'un graphe **non orienté** à partir de plusieurs parcours.

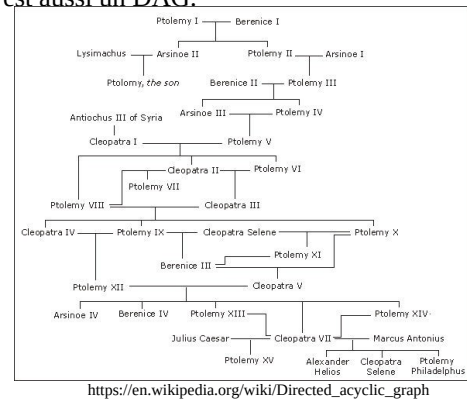
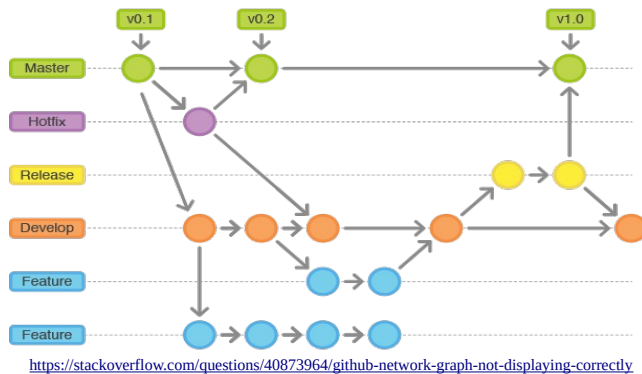
Voici des exemples de forêts obtenues avec des graphes de 500 sommets pour différentes valeurs de sigma (et des parcours en largeur).



## TP – Graphes orientés acycliques

Temps prévu pour ce TP : 4h.

Un graphe orienté est dit acyclique s'il ne contient pas de cycle. Intuitivement, un DAG (Directed Acyclic Graph = graphe orienté acyclique) est composé de sommets « source » sans arête entrante, de sommets « puit » sans arête sortante et de sommets internes qui permettent de faire le chemin entre les sommets « source » et les sommets « puit ». Les DAG sont des objets omniprésents dans plusieurs domaines comme la généalogie, la science des données, la planification ou encore l'informatique. Les images ci-dessous (trouvées sur internet) représentent les DAG d'un dépôt Git ou le DAG d'une généalogie. Le graphe au début de ce cahier de TP est aussi un DAG.



L'objectif de ce TP est de résoudre plusieurs problèmes classiques sur les DAG (complétez le fichier source `topo.c` dans le répertoire `dag`). Vous vous appuyerez sur la structure de graphe du précédent TP (Graph).

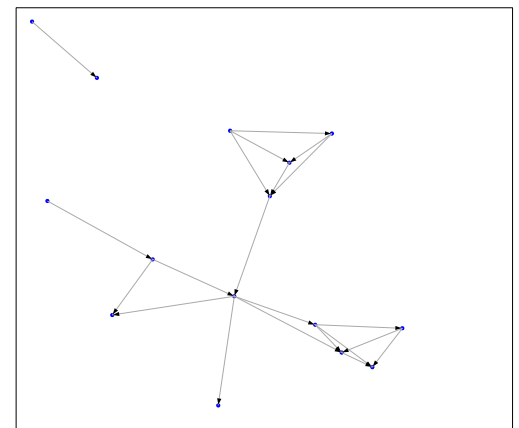
### Générer des DAG

On peut générer un DAG à partir d'un ensemble de points dans  $[0,1]^2$ . Il existe une arête  $(P_1, P_2)$  du point  $P_1(x_1, y_1)$  vers le point  $P_2(x_2, y_2)$  si

- la distance entre  $P_1$  et  $P_2$  est plus petite qu'un paramètre  $\sigma$  donné
- et  $y_1 < y_2$ .

La figure ci-contre a été obtenue de cette manière et présente un DAG avec 3 source et 4 puits.

**Travail à réaliser :** Normalement, générer un DAG a été réalisé dans le précédent TP sur les graphe. Si nécessaire, vérifiez et adaptez.



### Ordonner les sommets

Avec le cours, vous disposez de tous les éléments théoriques pour résoudre le problème suivant.

**Problème :** étant donné un DAG à  $n$  sommets, construire une liste ordonnée de sommets  $(x_1, x_2, \dots, x_n)$  telle que pour tout couple de sommets  $(x_i, x_j)$ , si  $x_i$  est un prédécesseur  $x_j$ , alors  $i < j$ .

Une telle liste est appelée un ordre topologique du DAG. Une conséquence directe de l'ordre topologique est que le premier sommet de l'ordre est une source alors que le dernier est un puits.

**Travail à réaliser :** écrivez une fonction `topologicalSort(Graph graph)` qui résout le problème et qui stockera la solution dans le tableau `topological_ordering`. Faites un affichage console de la solution.

Attention pour résoudre ce problème, il est explicitement demandé de ne pas utiliser les coordonnées des points (un simple tri suffirait). Il faut s'appuyer sur la représentation par listes d'adjacence.

## Dates au plus tôt

On se place ici dans la cadre d'un DAG pondéré  $G=(V,E,w)$  avec  $V$  l'ensemble des sommets,  $E$  l'ensemble des arêtes et  $w$  la fonction de pondération. Dans ce TP, la pondération d'une arête sera naturellement donnée par la distance euclidienne entre les deux sommets qui composent l'arête.

Ce type de DAG modélise par exemple un ensemble de tâches à réaliser avec des contraintes de précédence et un temps d'exécution pour chaque tâche.

**Définition :** la date au plus tôt d'un sommet source (sans arête entrante) est fixée à 0. La date au plus tôt d'un sommet  $s$  non source (avec des arêtes entrantes) est donnée par :

$$dtot(s) = \max(dtot(s') + w(s', s); s' \text{ prédécesseur de } s)$$

Si chaque sommet représente des travaux à réaliser, la date au plus tôt d'un sommet est la date optimale du début des travaux correspondant au sommet.

**Travail à réaliser :** écrivez une fonction `[++] computeEarliestStartDates(Graph graph)` qui résout le problème et qui stocke la solution dans le tableau `earliest_start`. Faites un affichage console de la solution.

## Dates au plus tard

On se place dans le même cadre que la partie précédente.

**Définition :** la date au plus tard d'un sommet puits (sans arête sortante) est fixée au maximum des dates au plus tôt des sommets puits. La date au plus tard d'un sommet  $s$  non puits (avec des arêtes sortantes) est donnée par :

$$dtard(s) = \min(dtard(s') - w(s, s'); s' \text{ successeur de } s)$$

Si chaque sommet représente des travaux à réaliser, la date au plus tard d'un sommet est la date maximale à laquelle les travaux correspondant au sommet doivent commencer pour ne pas prendre de retard.

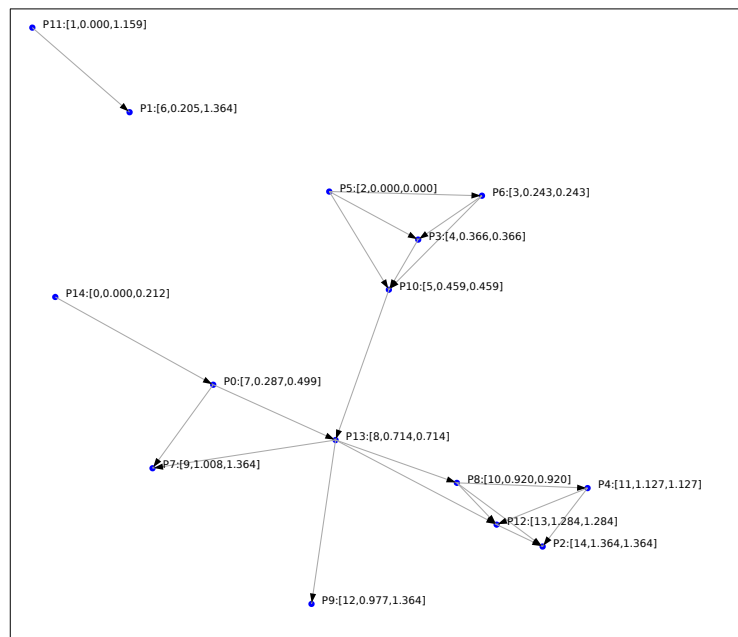
**Travail à réaliser :** écrivez une fonction `computeLatestStartDates(Graph graph)` qui résout le problème et qui stockera la solution dans le tableau `latest_start`. Faites un affichage console de la solution.

```

Print graph:
Vertex 0: (0.316913,0.594309)
Vertex 1: (0.183619,0.159360)
Vertex 2: (0.841916,0.852918)
Vertex 3: (0.644645,0.362065)
Vertex 4: (0.914059,0.759726)
Vertex 5: (0.502944,0.286326)
Vertex 6: (0.745625,0.291710)
Vertex 7: (0.221577,0.727858)
Vertex 8: (0.706567,0.750446)
Vertex 9: (0.473889,0.943460)
Vertex 10: (0.597800,0.442557)
Vertex 11: (0.029429,0.024232)
Vertex 12: (0.769144,0.818098)
Vertex 13: (0.512418,0.683061)
Vertex 14: (0.065862,0.454250)
Vertex 0: 13 -> 7 -> NULL
Vertex 1: NULL
Vertex 2: NULL
Vertex 3: 10 -> NULL
Vertex 4: 12 -> 2 -> NULL
Vertex 5: 10 -> 6 -> 3 -> NULL
Vertex 6: 10 -> 3 -> NULL
Vertex 7: NULL
Vertex 8: 12 -> 4 -> 2 -> NULL
Vertex 9: NULL
Vertex 10: 13 -> NULL
Vertex 11: 1 -> NULL
Vertex 12: 2 -> NULL
Vertex 13: 12 -> 9 -> 8 -> 7 -> NULL
Vertex 14: 0 -> NULL
Topological ordering:
[14 11 5 6 3 10 1 0 13 7 8 4 9 12 2 ]
Print start dates:
vertex 0 : [earliest start date=0.29, latest start date=0.50 ]
vertex 1 : [earliest start date=0.21, latest start date=1.36 ]
vertex 2 : [earliest start date=1.36, latest start date=1.36 ]
vertex 3 : [earliest start date=0.37, latest start date=0.37 ]
vertex 4 : [earliest start date=1.13, latest start date=1.13 ]
vertex 5 : [earliest start date=0.00, latest start date=0.00 ]
vertex 6 : [earliest start date=0.24, latest start date=0.24 ]
vertex 7 : [earliest start date=1.01, latest start date=1.36 ]
vertex 8 : [earliest start date=0.92, latest start date=0.92 ]
vertex 9 : [earliest start date=0.98, latest start date=1.36 ]
vertex 10 : [earliest start date=0.46, latest start date=0.46 ]
vertex 11 : [earliest start date=0.00, latest start date=1.16 ]
vertex 12 : [earliest start date=1.28, latest start date=1.28 ]
vertex 13 : [earliest start date=0.71, latest start date=0.71 ]
vertex 14 : [earliest start date=0.00, latest start date=0.21 ]

```

Un affichage console possible



Un affichage graphique possible : entre crochets  
[indice ordre topologique, date au plus tôt, date au plus tard]

## TP – Arbres couvrants minimums

Temps prévu pour ce TP : 3h.

L'objectif de ce TP est d'implémenter l'algorithme de Prim vu en cours pour calculer un arbre couvrant minimum (ACM) d'un graphe non-orienté et pondéré (répertoire `spanningtree`). L'algorithme de Prim s'appuie sur la structure de file de priorité qu'il faut au préalable implémenter (`Heap`, voir précédents TPs).

Comme pour les précédents TP, un `makefile` et des fichiers de tests naïfs sont fournis dans le répertoire `spanningtree`. Il suffit de faire `make all` puis `./testprim` pour tester vos développements.

### Algorithme de Prim

L'algorithme de Prim a été présenté en cours. Il s'appuie sur la structure de tas pour construire un arbre couvrant minimum d'un graphe non orienté et pondéré à partir d'un sommet. Si le graphe est composé de plusieurs composantes connexes, ce n'est pas un arbre mais une forêt couvrante minimale qui est calculée.

Nous reprenons la structure de graphe (`Graph`) non-orientée des précédents TPs. Les graphes seront construits à partir de points tirés au hasard dans  $[0,1] \times [0,1]$ . Les sommets du graphe seront les points tirés au hasard. Une arête  $\{A,B\}$  avec  $A=(x,y)$  et  $B=(x',y')$  sera présente dans le graphe si la distance entre A et B est plus petite qu'un seuil  $\sigma$  fixé dans  $[0,1]$ .

A la différence des précédents TPs, le poids d'une arête sera donnée par la distance euclidienne entre le deux sommets de l'arête.

#### Travail à faire :

- Implémentez une fonction `[++] void Prim(Graph g, int s)` qui calcule une forêt couvrante minimum du graphe `g` avec l'algorithme de Prim en utilisant la représentation par tableau de listes de successeurs. La forêt sera stockée dans le champs `parents` de la structure de graphe.

